ADVANCED SOFTWARE ENGINEERING (LAB) — UNIVERSITÀ DI PISA

Monster Tamers

Riepilogo della consegna e dei requisiti funzionali

Docente: Alessandro Bocci

Fonti: *Lab Project Kick-off* (07/10/2026) e *Monster_Tamers_Backlog.xlsx*

In breve

- **Cosa:** un gioco di collezione di mostri (*Monster Tamers*) realizzato come **applicazione distribuita** con backend **REST**, uno o più **database** e un **frontend minimale**.
- **Chi:** gruppi da **3 o 4 persone**, da formare entro il **1 novembre**.
- **Scadenza finale: 15 dicembre**. Discussione orale a **gennaio/febbraio**.
- **Cosa si consegna:** repository Git (codice + documentazione) e un breve report su template fornito dal docente.
- **Requisiti:** 28 user stories — **18** per la prima consegna, **2** in più per la consegna finale, **8** opzionali.

Indice

1 Organizzazione e scadenze **2**

2 Cosa consegnare **2**

3 Requisiti tecnici del software **5**

4 Il gioco: Monster Tamers **6**

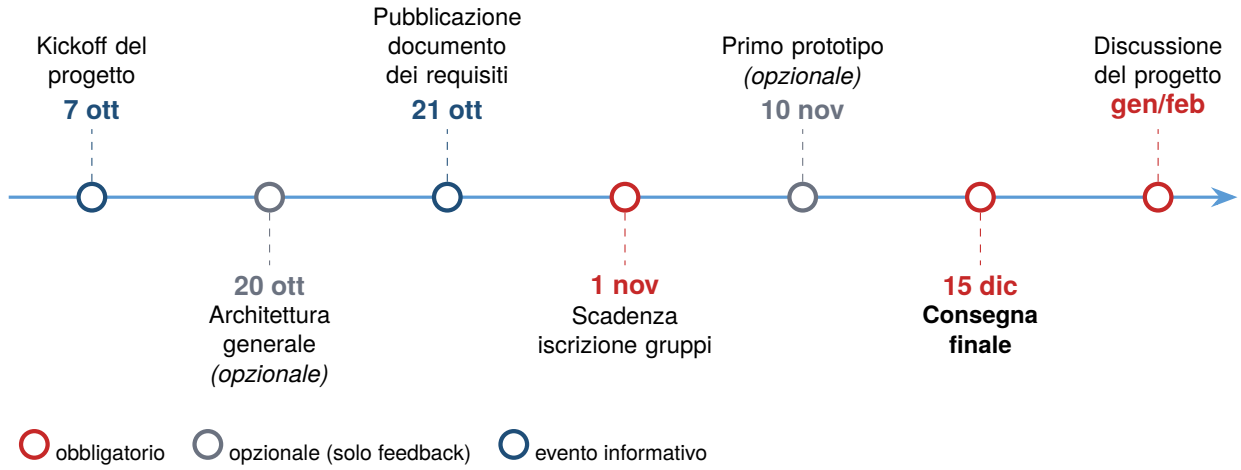
5 Requisiti funzionali (user stories) **7**

1 Organizzazione e scadenze

1.1 Gruppi

- Gruppi da **3 o 4 persone**.
- Per formarli si usa il **documento condiviso su Moodle**: seguire le istruzioni del primo foglio per cercare compagni e “iscrivere” il gruppo.
- **1 novembre**: la modifica del file viene bloccata, quindi il gruppo deve essere iscritto entro quella data.

1.2 Roadmap



Data	Evento	Note
7 ottobre	Kickoff	Presentazione del progetto.
20 ottobre	Consegna architettura generale	**Opzionale**. Dettagli nella Sezione 2.1.
21 ottobre	Pubblicazione requisiti	Il docente pubblica il documento con i requisiti del progetto.
1 novembre	Scadenza gruppi	Il documento condiviso su Moodle viene bloccato.
10 novembre	Consegna primo prototipo	**Opzionale**. Dettagli nella Sezione 2.2.
15 dicembre	**Consegna finale**	**Obbligatoria**. Dettagli nella Sezione 2.3.
gennaio e febbraio	Discussione	Orale di gruppo con valutazione individuale (Sezione 2.4).

1.3 Perché ci sono consegne opzionali

> Le consegne opzionali servono **solo a ricevere feedback**: **non hanno voto** e **non c’è penalità** se non si consegna. Sono un’occasione per scoprire presto problemi di progettazione, perché un feedback anticipato costa di solito meno di un refactoring successivo. Se non si consegna l’architettura non succede nulla, si perde solo il feedback anticipato.

In caso di dubbi o blocchi si può scrivere al docente via email e/o andare a ricevimento.

2 Cosa consegnare

2.1 Consegna 1 — Architettura generale OPZIONALE · 20 OTT

Uno **schizzo di un'architettura software monolitica** che contenga:

1. i **componenti principali**;
2. la **comunicazione con gli utenti** (quali operazioni fa l'utente e verso quale componente);
3. il **contenuto del/i database**;
4. le **user stories complete** (con criteri di accettazione e dipendenze, vedi Sezione 5.2).

Formato

File **.pdf di slide** (con l'account Unipi sono disponibili gratis Office e Google Docs), possibilmente **sotto le 10 slide**, caricato su **Moodle** (sezione progetto) da **un solo membro** del gruppo.

Slide	Contenuto
1	Nome del gruppo e membri.
2	Rappresentazione delle informazioni del giocatore.
3	Collezione dei mostri e regole degli *encounter*.
4 (max 2 slide)	User stories complete.
successive	Disegno dell'architettura, interazioni dell'utente e database.

Linee guida per disegnare l'architettura

- Partire dalle **user stories** per individuare le operazioni degli utenti.
- Definire **componenti con uno scopo chiaro**.
- Ricordare che poi andranno **implementati i componenti e la API RESTful**.
- **KISS** — *Keep It Simple and Stupid*.

Esempio del docente: negozio online (riassunto)

L'esempio mostra il livello di dettaglio atteso. Ogni operazione dell'utente è associata a un componente e a un endpoint REST (descrivibile con **OpenAPI**); ogni componente che deve ricordare dati ha il suo database.

Componente	Operazioni utente	Endpoint REST	DB
Authentication	login	`PUT /login?user={username}...`	Users
Catalogue	consultare gli articoli	`GET /items`, `GET /items/{item_id}`	Items
Orders	creare ordini e pagare, controllare lo stato	`PUT /orders` + JSON, `PUT /payment` + JSON, `GET /orders/{id_order}`	Orders
Payments	interno (verso un *Bank service* esterno)	—	—

Errore tipico evidenziato: nella prima versione dell'esempio l'operazione "controllare lo stato dell'ordine" non aveva un database degli ordini su cui appoggiarsi. Ogni operazione deve avere i dati che le servono.

Database: i DB disegnati possono essere DBMS diversi, database diversi nello stesso DBMS oppure tabelle diverse dello stesso DB. La scelta è libera.
Record d'esempio: *Users* (username, password, dati di fatturazione, storico, …), *Items* (nome, prezzo, disponibilità, …).

2.2 Consegna 2 — Primo prototipo OPZIONALE · 10 NOV

Un **repository GitHub** contenente:

- **descrizione generale** del progetto;
- **codice**;
- **quick start**: istruzioni di installazione ed esecuzione;
- **API completa e relativi test**.

Come preparare quick start, API e test verrà spiegato nei prossimi laboratori. Il prototipo copre le user stories PRIMA CONSEGNA.

2.3 Consegna finale OBBLIGATORIA · 15 DIC

- **Repository GitHub** strutturato come per il prototipo (codice e documentazione).
- **Report** che descrive il progetto, scritto sul **template fornito dal docente**.

La differenza tra prototipo e consegna finale è l'aggiunta delle funzionalità (requisiti) che verranno trattate più avanti nel corso: in particolare le user stories CONSEGNA FINALE.

2.4 Discussione del progetto (gennaio/febbraio)

- La consegna finale riceve **lo stesso voto per tutti i membri** del gruppo.
- Nel periodo d'esame (gennaio e febbraio) il docente proporrà alcune date per incontrare il gruppo.
- Durante la discussione il docente fa domande sulla realizzazione del progetto per valutare il **coinvolgimento individuale**.
- La discussione **aggiunge o toglie punti** al voto della consegna, **individualmente**.

> Se uno studente non sa rispondere e non dimostra un coinvolgimento sufficiente nel progetto, il suo voto finale è **"Failed"** e dovrà sostenere l'esame **senza il progetto**.

3 Requisiti tecnici del software

Applicazione distribuita

Il sistema deve essere un'applicazione distribuita.

Interfaccia REST

Tutte le funzionalità devono essere utilizzabili tramite un'interfaccia RESTful.

Uno o più database

Persistenza dei dati, incluso lo storico degli *encounter*.

Meccanismi di sicurezza

Protezione di account e dati di gioco.

Testing

Test unitari e test di performance.

Analisi di sicurezza

Analisi della sicurezza del sistema.

3.1 Backend

Il focus principale del progetto è il **backend**. Il servizio deve gestire:

- la **collezione globale** dei mostri e le **collezioni personali** dei giocatori;
- la **selezione del team**;
- gli ***encounter***;
- lo **storico degli *encounter***, con **memorizzazione permanente**.

Tutto deve essere eseguibile tramite l'**interfaccia RESTful**.

3.2 Frontend

- Richiesto un **frontend minimale** con una UI di base, usato come sistema di debug, che chiama la **REST API del backend** per le operazioni principali.
- L'aspetto grafico è libero: conta come viene usato e come interagisce con il backend.

> **Nessuna logica di business rilevante nel frontend.** Le regole del gioco stanno nel backend.

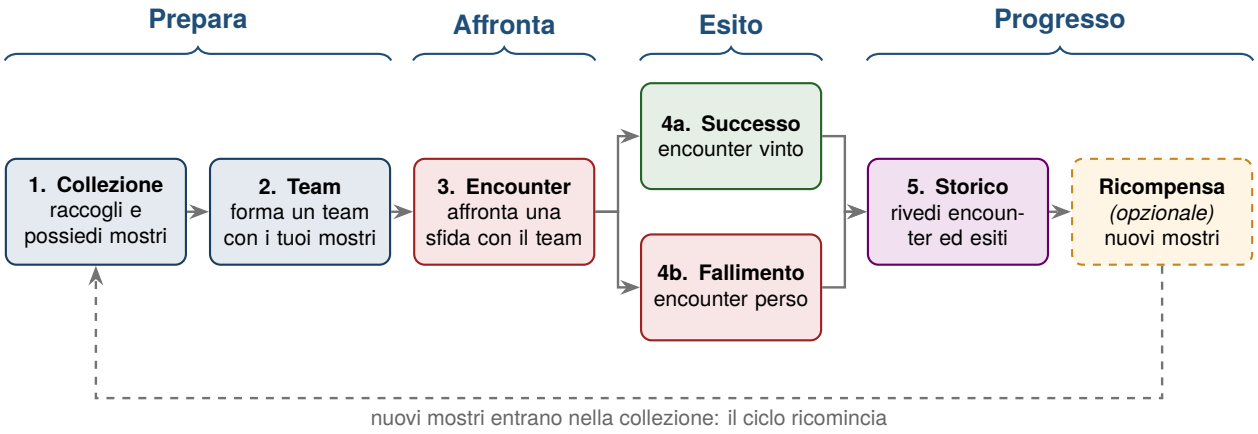
3.3 Scelte libere

Linguaggi di programmazione, database e altri aspetti non specificati sono a scelta del gruppo. Le funzionalità da implementare sono quelle descritte nelle user stories (Sezione 5).

4 Il gioco: Monster Tamers

4.1 Il ciclo di gioco (game loop)

Il ciclo minimo richiesto è **Prepara** → **Affronta** → **Esito** → **Progresso**. Si può estendere a piacere.



4.2 Giocatori

Come rappresentare un giocatore è a scelta del gruppo. **Minimo obbligatorio:**

- delle **credenziali**;
- lo **storico degli *encounter***.

Si possono aggiungere altri dati (immagine, amici, …) e si possono tenere separati **profilo** e **account** del giocatore.

4.3 Mostri

- Ogni gruppo sceglie la **collezione di mostri** del proprio progetto.
- Ogni mostro deve avere **almeno**:
  - un'**immagine** che lo rappresenta;
  - **due caratteristiche** usate nel gameplay degli *encounter* (se ne possono aggiungere altre).
- Le caratteristiche **influenzano gli *encounter***.
- Ogni giocatore possiede una **collezione personale**.
- Durante il gioco si possono **ottenere nuovi mostri**.

> **Solo come esempio**, nelle slide i mostri hanno: un elemento (fuoco, acqua, terra, oscurità), le statistiche *Power*, *Speed*, *Defense*, *Magic* (da 1 a 5 stelle) e un'abilità speciale (es. "riduce il danno subito del 25%"). Non è un vincolo.

4.4 Team

Prima di un *encounter* il giocatore prepara un team con i mostri della propria collezione. Il team deve:

- avere una **dimensione limitata**;
- **rispettare le regole del gioco**;
- essere composto **solo da mostri posseduti**.

Le regole precise dei team sono a scelta del gruppo.

4.5 Encounter

Un *encounter* è una sfida tra un team e un **"mostro selvatico"**.

- *encounter* diversi possono avere caratteristiche diverse;
- il risultato **dipende dal team scelto**;
- un *encounter* può **riuscire o fallire**;
- alcuni esiti possono **dare un nuovo mostro**.

Un *encounter* **non deve per forza** essere una battaglia a turni.

4.6 Regole del gioco

Le regole sono in gran parte libere. Ad esempio vanno definiti: l'obiettivo principale del gioco, come si amplia la collezione, come funzionano gli *encounter*, se sono ammessi duplicati nelle collezioni personali, ecc.

Esempio dalle slide: encounter basati su combattimenti, con l'obiettivo di battere un boss finale dopo diverse battaglie di difficoltà crescente.

Decisioni lasciate al gruppo

- Dati del giocatore oltre al minimo (credenziali + storico); profilo e account separati o no.
- Collezione dei mostri: quali mostri, quali caratteristiche (almeno immagine + 2 caratteristiche).
- Regole dei team: dimensione massima e vincoli.
- Meccanica degli *encounter*: come si calcola l'esito (anche non a turni), quando si ottiene un mostro.
- Obiettivo del gioco, crescita della collezione, gestione dei duplicati.
- Funzionalità extra (user stories verdi o idee nuove).
- Linguaggi, database, aspetto grafico del frontend.

5 Requisiti funzionali (user stories)

5.1 Come leggere le priorità

Colore	Significato	N. US
PRIMA CONSEGNA	Funzionalità core, da implementare già nel **primo prototipo** (e quindi presenti anche nella consegna finale).	**18**
CONSEGNA FINALE	Funzionalità da aggiungere per la **consegna finale** (legate ad argomenti trattati più avanti nel corso).	**2**
OPZIONALE	**Facoltative**: proposte dagli studenti, si possono implementare oppure no.	**8**

- La priorità indica il **prodotto minimo funzionante per il progetto**, non per un prodotto reale.
- Le user stories sono volutamente **generiche** per lasciare libertà ai gruppi (ad esempio, non esiste una user story su come implementare gli *encounter*). Quelle specifiche sono tutte verdi.

- Si possono aggiungere **funzionalità non presenti** nelle user stories.
- In tutte le user stories l'attore è **il giocatore** ("Come giocatore, voglio…").

Nota: il file Excel contiene anche un foglio "Realistic backlog" con priorità P1–P3 pensate per un gioco reale. Non riguardano il progetto e sono state omesse; da quel foglio sono ripresi solo i titoli brevi delle user stories.

5.2 Da completare a cura del gruppo

Il file del docente riporta solo le user stories con la loro priorità (criteri di accettazione e dipendenze sono stati omessi di proposito). Il docente chiede di:

- definire i **criteri di accettazione** almeno per le user stories ad **alta e media priorità**;
- definire le **dipendenze** tra le user stories.

5.3 Account e progressi

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-01	**Creare un account**	Voglio creare un account, *così che* i miei progressi di gioco siano associati a me.	PRIMA CONSEGNA
US-02	**Accedere (login)**	Voglio effettuare il login, *così che* io possa accedere al mio gioco e continuare a giocare.	PRIMA CONSEGNA
US-03	**Ritrovare i progressi salvati**	Voglio ritrovare i miei progressi quando torno, *così che* io possa riprendere da dove avevo lasciato.	PRIMA CONSEGNA
US-04	**Personalizzare il profilo**	Voglio scegliere o modificare il mio nome da giocatore e il mio avatar, *così che* io possa personalizzare la mia identità.	OPZIONALE
US-05	**Proteggere i dati dell'account**	Voglio che i dati del mio account siano protetti, *così che* nessun altro possa accedervi o modificarli senza permesso.	CONSEGNA FINALE

5.4 Collezione di mostri

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-06	**Ricevere una collezione iniziale**	Voglio ricevere un set iniziale di mostri quando inizio a giocare, *così che* io possa formare il mio primo team.	PRIMA CONSEGNA
US-07	**Scegliere il primo mostro**	Voglio scegliere il mio primo mostro, *così che* io possa personalizzare la mia strategia iniziale.	OPZIONALE
US-08	**Scegliere una fazione**	Voglio scegliere una fazione (o un'affiliazione iniziale equivalente), *così che* la mia scelta iniziale possa influenzare l'esperienza di gioco.	OPZIONALE
US-09	**Vedere la collezione personale**	Voglio vedere i mostri che possiedo, *così che* io sappia quali mostri posso usare.	PRIMA CONSEGNA
US-10	**Sfogliare la collezione globale**	Voglio sfogliare tutti i mostri disponibili nel gioco, *così che* io capisca cosa posso incontrare od ottenere.	PRIMA CONSEGNA
US-11	**Vedere i dettagli di un mostro**	Voglio vedere l'immagine e le caratteristiche di un mostro, *così che* io possa valutare quanto può essermi utile negli *encounter*.	PRIMA CONSEGNA
US-12	**Ottenere un mostro guadagnato**	Voglio che un mostro guadagnato o catturato in un *encounter* venga aggiunto alla mia collezione, *così che* le mie scelte future si amplino.	PRIMA CONSEGNA
US-13	**Sviluppare i mostri**	Voglio migliorare o personalizzare i miei mostri, *così che* io possa sviluppare strategie diverse nel tempo.	OPZIONALE

5.5 Team

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-14	**Creare un team**	Voglio creare un team usando i mostri della mia collezione, *così che* io possa prepararmi a un *encounter*.	PRIMA CONSEGNA
US-15	**Modificare un team**	Voglio sostituire o rimuovere mostri dal mio team, *così che* io possa adattare la strategia prima di un *encounter*.	PRIMA CONSEGNA
US-16	**Validare un team**	Voglio sapere se il team selezionato rispetta le regole del gioco, *così che* io non inizi con un team non valido.	PRIMA CONSEGNA

5.6 Encounter

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-17	**Sfogliare gli encounter**	Voglio vedere gli *encounter* disponibili e le loro caratteristiche, *così che* io possa scegliere una sfida adatta.	PRIMA CONSEGNA
US-18	**Avviare un encounter**	Voglio avviare un *encounter* con il team selezionato, *così che* io possa affrontare una sfida.	PRIMA CONSEGNA
US-19	**Risolvere un encounter**	Voglio che l'*encounter* venga risolto secondo le regole del gioco, *così che* io sappia se ho avuto successo o ho fallito.	PRIMA CONSEGNA
US-20	**Dettagli del risultato**	Voglio vedere i dettagli del risultato di un *encounter*, *così che* io capisca l'esito e il team coinvolto.	PRIMA CONSEGNA
US-21	**Abbandonare un encounter**	Voglio abbandonare o fuggire da un *encounter* quando le regole lo consentono, *così che* io possa interrompere una sfida che non voglio più continuare.	OPZIONALE
US-22	**Scegliere le azioni**	Voglio scegliere azioni come attaccare o tentare di catturare un mostro durante un *encounter*, *così che* le mie decisioni possano influenzarne l'esito.	OPZIONALE

5.7 Storico e integrità

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-23	**Vedere lo storico**	Voglio vedere i miei *encounter* precedenti, *così che* io possa rivedere i miei progressi e i risultati.	PRIMA CONSEGNA
US-24	**Dettagli di un encounter passato**	Voglio vedere i dettagli di un *encounter* passato, *così che* io possa imparare dalla scelta del team e dal risultato.	PRIMA CONSEGNA
US-25	**Proteggere i salvataggi**	Voglio che collezione, storico e progressi siano protetti da manomissioni o perdite, *così che* io possa fidarmi del mio salvataggio.	CONSEGNA FINALE

5.8 Social

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-26	**Gestire gli amici**	Voglio aggiungere e gestire amici, *così che* io possa tenere traccia delle persone che conosco nel gioco.	OPZIONALE
US-27	**Sfidare un amico**	Voglio sfidare un amico, *così che* io possa giocare con altri in una modalità aggiuntiva.	OPZIONALE

5.9 Interfaccia del giocatore

ID	Titolo	Cosa vuole il giocatore e perché	Quando
US-28	**Usare un'interfaccia semplice**	Voglio eseguire le azioni principali del gioco tramite un'interfaccia semplice, *così che* io possa giocare senza usare strumenti tecnici.	PRIMA CONSEGNA

5.10 Vista per consegna

Primo prototipo (18)

US-01 Creare account
US-02 Login
US-03 Ritrovare progressi
US-06 Collezione iniziale
US-09 Collezione personale
US-10 Collezione globale
US-11 Dettagli mostro
US-12 Ottenere mostro
US-14 Creare team
US-15 Modificare team
US-16 Validare team
US-17 Sfogliare encounter
US-18 Avviare encounter
US-19 Risolvere encounter
US-20 Dettagli risultato
US-23 Storico
US-24 Dettagli encounter passato
US-28 Interfaccia semplice

+ Consegna finale (2)

US-05 Protezione dati account
US-25 Protezione salvataggi da manomissioni o perdite

Tutte e due riguardano la sicurezza e l'integrità dei dati, in linea con i requisiti tecnici "meccanismi di sicurezza" e "analisi di sicurezza".

La consegna finale comprende quindi tutte le 18 US rosse più queste 2.

Opzionali (8)

US-04 Nome e avatar
US-07 Scegliere primo mostro
US-08 Scegliere fazione
US-13 Sviluppare mostri
US-21 Abbandonare encounter
US-22 Scegliere azioni
US-26 Gestire amici
US-27 Sfidare amico